

Escopo crud-Tutorial

Data de criação: 16/09

Ultima Atualização: 17/09

Stack tecnológica

O sistema possui um backend desenvolvido em Java com Spring Boot e terá um frontend em Vue.js. A interface permitirá realizar as operações disponíveis na API, utilizando como referência os protótipos elaborados no Figma.

Backend

- Java 21
- Spring Boot 4.1.1
- Spring Web/Spring MVC
- Spring DataJPA
- JPA e Hibernate
- PostgreSQL
- MapStruct 1.6.3
- Maven
- JUnit, Mockito e MockMvc

Vue.js é a tecnologia definida para a futura interface. As demais escolhas do frontend constituem uma proposta inicial e poderão ser ajustadas durante a implementação.

A integração seguirá o fluxo:

Interface Vue.js → requisição HTTP → API Spring Boot → serviços e repositórios → PostgreSQL.

Após o processamento, a API devolverá uma resposta ao frontend, geralmente em JSON, para atualizar a interface ou apresentar mensagens de resultado. O acesso ao banco de dados permanecerá sob responsabilidade do backend.

Objetivos

O projeto tem como objetivo disponibilizar uma API REST para centralizar o gerenciamento de tutoriais e informações relacionadas, permitindo que outras aplicações solicitem operações de cadastro, consulta, atualização e exclusão por meio de requisições HTTP. A implementação busca organizar o processamento dessas operações, o armazenamento dos dados e os relacionamentos entre tutoriais, autores, cursos e detalhes complementares.

A base tecnológica utiliza Java 21 e Spring Boot, com o Spring Web disponibilizando os pontos de acesso da API e produzindo as respostas às requisições. O Maven gerencia as

dependências e o processo de construção do projeto. A persistência utiliza Spring Data JPA e Hibernate para relacionar os objetos Java às tabelas do banco PostgreSQL, reduzindo a necessidade de implementar manualmente as operações mais comuns de acesso aos dados.

o código segue uma arquitetura em camadas, organizada em: `Controllers`, `Services` e `Repositories`

Os controllers recebem as requisições, encaminham os dados aos serviços e devolvem as respostas com os códigos HTTP correspondentes. O `TutorialController` concentra operações relacionadas aos tutoriais, enquanto o `RelationshipController` disponibiliza operações envolvendo autores, cursos, detalhes e suas associações. A camada de serviços coordena o processamento das solicitações. As interfaces `TutorialService` e `RelationshipService` definem as operações disponíveis, e as classes `TutorialServiceImpl` e `RelationshipServiceImpl` implementam seus comportamentos. Entre suas responsabilidades estão verificar a existência dos registros, definir que novos tutoriais sejam inicialmente cadastrados como não publicados e coordenar a criação ou remoção de associações. As dependências entre controllers, serviços, repositórios e mapeadores são fornecidas pelo Spring por meio de injeção de dependências via construtor, facilitando a organização e os testes de cada componente.

A camada de persistência adota o padrão Repository por meio de interfaces que estendem `JpaRepository`. Esses repositórios disponibilizam operações de armazenamento, consulta e exclusão, além de buscas específicas definidas no projeto. As entidades `Tutorial`, `Author`, `Course` e `TutorialDetail` representam os dados persistidos e seus relacionamentos: um autor pode estar associado a vários tutoriais; cursos e tutoriais possuem uma associação de muitos para muitos; e cada tutorial pode possuir um registro de detalhes, contendo informações como conteúdo e tempo estimado.

Para transportar os dados entre a API e a camada de serviços, o projeto utiliza o padrão DTO, separando os objetos de entrada e saída das entidades de persistência. A conversão entre essas representações é realizada por mapeadores definidos com `MapStruct`, que gera as implementações durante a compilação. O fluxo de uma solicitação envolve, portanto, o recebimento dos dados pelo controller, o processamento pelo serviço, o acesso ao banco pelo repository e, quando necessário, a conversão do resultado em DTO para compor a resposta. O projeto também contém testes de controllers, serviços e mapeamentos, utilizando recursos como JUnit, Mockito e MockMvc para verificar comportamentos dessas partes da aplicação.

O que o sistema permite fazer:

- o sistema atual permite cadastrar, consultar, atualizar e excluir tutoriais (CRUD).
- permite registrar se um tutorial está publicado.
- cadastrar autores e associá-los aos tutoriais.
- cadastrar cursos e associar tutoriais a eles.
- adicionar informações complementares aos tutoriais.

Elemento	O que a implementação permite
Tutoriais	Cadastrar, consultar, atualizar, excluir e filtrar registros, além de registrar a situação de publicação.
Autores	Cadastrar, consultar por identificador e associar um autor a um tutorial.
Cursos	Cadastrar, consultar por identificador e associar ou desassociar tutoriais.
Detalhes dos Tutoriais	Registrar e consultar informações complementares de um tutorial.

Com base nessas capacidades atuais, propõe-se o GOTS (Gerenciamento e Organização de Tutoriais Simples) como um catálogo destinado a pessoas responsáveis por cadastrar e organizar conteúdos instrucionais. A API existente oferece a base para reunir tutoriais, identificar seus autores e associá-los a cursos, favorecendo a consulta e a manutenção das informações. A proposta prioriza a simplicidade de organização dos conteúdos, tendo como ponto de partida as funcionalidades já implementadas.

Requisitos funcionais

Código	Requisito	Descrição
RF01	Cadastrar tutorial	O sistema deve permitir cadastrar um tutorial com título e descrição, gerando automaticamente seu identificador e definindo sua situação inicial como não publicado.
RF02	Listar tutoriais	O sistema deve permitir consultar os tutoriais cadastrados, retornando os resultados de forma paginada.
RF03	Consultar tutorial por id	O sistema deve permitir consultar um tutorial específico pelo seu identificador.
RF04	Filtrar tutoriais por título	O sistema deve permitir filtrar a listagem de tutoriais por um trecho do título informado na consulta.
RF05	Buscar por título	O sistema deve disponibilizar uma consulta por trecho do título
RF06	Consultar tutoriais publicados	O sistema deve permitir consultar, de forma paginada, os tutoriais cuja situação de publicação esteja marcada como publicada.
RF07	Atualizar tutorial	O sistema deve permitir atualizar o título, a descrição e a situação de publicação de um tutorial existente, preservando seu identificador. As associações com autores,

		cursos e detalhes devem ser tratadas pelas operações específicas.
RF08	Excluir tutorial	O sistema deve permitir excluir um tutorial pelo seu identificador. Os detalhes vinculados devem acompanhar a exclusão, sem excluir os cadastros de autores e cursos associados.
RF09	Excluir todos os tutoriais.	O sistema deve disponibilizar uma operação para excluir todos os tutoriais cadastrados.
RF10	Controlar paginação e ordenação	Nas listagens de tutoriais, o sistema deve permitir informar a página, a quantidade de registros por página e a ordenação. Na ausência de uma ordenação informada, deve utilizar o identificador em ordem crescente.
RF11	Cadastrar autor	O sistema deve permitir cadastrar um autor com seu nome e gerar automaticamente seu identificador.
RF12	Consultar autor	O sistema deve permitir consultar um autor pelo identificador, incluindo os tutoriais associados a ele.
RF13	Associar autor a tutorial	O sistema deve permitir associar um autor existente a um tutorial existente. Cada tutorial deve possuir, no máximo, um autor associado, enquanto um autor pode estar associado a vários tutoriais. Uma nova associação deve substituir o autor anterior do tutorial.
RF14	Cadastrar curso	O sistema deve permitir cadastrar um curso com seu nome e gerar automaticamente seu identificador.
RF15	Consultar curso	O sistema deve permitir consultar os dados cadastrais de um curso pelo seu identificador.
RF16	Associar tutorial a curso	O sistema deve permitir associar um tutorial existente a um curso existente. Um tutorial pode pertencer a vários cursos, e um curso pode reunir vários tutoriais.
RF17	Remover associação entre tutorial e curso	O sistema deve permitir remover o vínculo entre um tutorial e um curso, preservando os dois cadastros. Caso o vínculo não exista, deve informar essa condição.
RF18	Registrar detalhes do tutorial	O sistema deve permitir registrar informações complementares para um tutorial existente, incluindo conteúdo e tempo estimado em minutos, mantendo uma relação de um para um entre tutorial e registro de detalhes.
RF19	Consultar detalhes do tutorial	O sistema deve permitir consultar os detalhes associados a um tutorial por meio do identificador do tutorial.

RF20	Verificar a existência dos registros	Nas operações que dependem de registros existentes, o sistema deve verificar sua existência e informar quando o tutorial, autor, curso ou registro de detalhes solicitado não for encontrado.
------	--------------------------------------	---

Requisitos não funcionais

Código	Requisito	Descrição
RNF01	Comunicação por API	O sistema deve disponibilizar suas operações por uma API REST baseada em HTTP, utilizando JSON nos corpos das requisições e respostas que transportam dados.
RNF02	Padronização das respostas	A API deve utilizar códigos HTTP coerentes com o resultado das operações, incluindo 200 para consultas e atualizações bem-sucedidas, 201 para criação de registros e 204 para exclusões ou remoções de vínculo concluídas sem conteúdo de resposta.
RNF03	Tratamento centralizado de erros	O sistema deve centralizar o tratamento de exceções no GlobalExceptionHandler e utilizar ProblemDetail para estruturar os erros tratados. Deve distinguir requisições inválidas, recursos não encontrados, conflitos de integridade e falhas internas por meio dos códigos HTTP 400, 404, 409 e 500, respectivamente.
RNF04	Registro de falhas internas	As falhas internas tratadas devem ser registradas nos logs da aplicação. A resposta enviada ao consumidor deve apresentar uma mensagem genérica, sem incluir a pilha de execução ou os detalhes internos da exceção.
RNF05	Consistência transacional	As operações de escrita devem utilizar transações para coordenar as alterações no banco. Nos métodos de serviço anotados com @Transactional, as operações participantes devem ser confirmadas em conjunto ou revertidas quando ocorrer uma falha que acione rollback.
RNF06	Separação de responsabilidades	O código deve manter uma arquitetura em camadas: controllers para receber e responder às requisições, services para coordenar as operações e repositories para acessar os dados persistidos.
RNF07	Organização das dependências	Controllers e serviços devem receber suas dependências por construtor, utilizando o gerenciamento de componentes do Spring. Os serviços devem possuir interfaces que definam

		suas operações, separadas das classes de implementação.
RNF08	Separação entre transporte e persistência	A API deve utilizar DTOs para representar os dados de entrada e saída, mantendo essas representações separadas das entidades de persistência. As conversões devem utilizar os mapeadores definidos com MapStruct.
RNF09	Persistência relacional	Os dados devem ser armazenados em PostgreSQL. O acesso deve utilizar Spring Data JPA, repositórios baseados em JpaRepository e mapeamento das entidades e de seus relacionamentos com JPA e Hibernate.
RNF10	Plataforma de implementação	O projeto deve utilizar Java 21 e Spring Boot, com Spring Web para implementar os pontos de acesso HTTP da aplicação.
RNF11	Gerenciamento da construção	O projeto deve utilizar Maven para declarar dependências, configurar a compilação e executar os testes. A geração das implementações dos mapeadores MapStruct deve ocorrer durante a compilação.
RNF12	Verificação automatizada	O projeto deve manter testes automatizados para comportamentos dos controllers, serviços e conversões entre entidades e DTOs, utilizando JUnit, Mockito e recursos de teste do Spring, incluindo MockMvc.

Histórias de usuário

User Stories para o responsável pelo catálogo.

Código	História de usuário	CrITÉRIOS de Aceitação	Descrição
US01 Cadastrar tutorial	Como responsável pelo catálogo, quero cadastrar um tutorial com título e descrição, para registrar um novo material.	1. O cadastro deve armazenar o título e a descrição enviados.2. O identificador deve ser gerado automaticamente.3. A situação inicial deve ser não publicado, mesmo que a requisição informe publicado.4. A API deve retornar HTTP 201 com os dados do tutorial criado.	Disponibiliza o cadastro inicial por meio de POST /api/tutorials. As associações com autores, cursos e detalhes são realizadas por operações específicas.

US02 Explorar o catálogo	Como responsável pelo catálogo, quero consultar os tutoriais em páginas e escolher sua ordenação, para examinar os registros em partes e na sequência desejada.	1. A consulta deve permitir informar a página e a quantidade de registros por página.2. Deve permitir informar a ordenação.3. Sem ordenação informada, os resultados devem seguir o identificador em ordem crescente.4. A resposta deve indicar se existe uma próxima página.	Disponibiliza a listagem por GET /api/tutorials. Utiliza Pageable e retorna Slice, sem exigir a contagem total dos registros.
US03 Consultar um tutorial	Como responsável pelo catálogo, quero consultar um tutorial pelo identificador, para verificar as informações de um registro específico.	1. A consulta deve localizar o tutorial pelo identificador informado.2. Quando encontrado, deve retornar HTTP 200 com seus dados.3. Quando não encontrado, deve retornar HTTP 404.	Disponibiliza a consulta individual por GET /api/tutorials/{id}, incluindo os cursos e os detalhes associados na representação do tutorial.
US04 Localizar tutoriais por título	Como responsável pelo catálogo, quero buscar tutoriais por um trecho do título, para localizar materiais sem precisar informar o título completo.	1. A listagem deve aceitar um filtro por trecho do título.2. A consulta /api/tutorials/by-title deve permitir essa busca sem diferenciar maiúsculas e minúsculas.3. Os resultados devem ser paginados.4. Uma busca sem correspondências deve retornar uma coleção vazia com HTTP 200.	Abrange o filtro title da listagem e a consulta específica por título. Apesar do nome interno findByExactTitle, a segunda consulta procura trechos do título.
US05 Consultar tutoriais publicados	Como responsável pelo catálogo, quero listar apenas os	1. A consulta deve retornar apenas tutoriais com published igual a	Disponibiliza GET /api/tutorials/published. A situação de publicação é um dado do cadastro e não representa, por si só, a disponibilização em uma página pública.

	tutoriais marcados como publicados, para identificar os materiais nessa situação.	true.2. Deve oferecer paginação e ordenação.3. Sem tutoriais publicados, deve retornar uma coleção vazia com HTTP 200.	
US06 Atualizar tutorial	Como responsável pelo catálogo, quero alterar os dados de um tutorial, para corrigir e manter suas informações atualizadas.	1. A atualização deve permitir alterar título, descrição e situação de publicação.2. Deve preservar o identificador e as associações existentes com autor, cursos e detalhes.3. Deve retornar HTTP 200 com os dados atualizados.4. Se o tutorial não existir, deve retornar HTTP 404.	Disponibiliza a atualização por PUT /api/tutorials/{id}. Os relacionamentos são administrados separadamente pelas operações específicas.
US07 Excluir um tutorial	Como responsável pelo catálogo, quero excluir um tutorial, para retirar um material que não precisa mais ser mantido.	1. O tutorial deve ser localizado pelo identificador antes da exclusão.2. A exclusão deve remover seus detalhes vinculados, preservando os cadastros de autores e cursos.3. Em caso de sucesso, deve retornar HTTP 204 sem corpo.4. Se o tutorial não existir, deve retornar HTTP 404.	Disponibiliza DELETE /api/tutorials/{id}. O relacionamento entre tutorial e detalhes possui exclusão em cascata.
US08 Esvaziar o catálogo de tutoriais	Como responsável pelo catálogo, quero excluir todos os tutoriais em uma operação, para	1. A operação deve excluir todos os tutoriais cadastrados.2. Deve preservar os cadastros de	Disponibiliza a exclusão coletiva por DELETE /api/tutorials. A implementação atual não possui uma etapa adicional de confirmação.

	reiniciar o conjunto de materiais cadastrados.	autores e cursos.3. Ao concluir, deve retornar HTTP 204 sem corpo.	
US09 Cadastrar autor	Como responsável pelo catálogo, quero cadastrar um autor, para associá-lo aos tutoriais correspondentes.	1. O cadastro deve armazenar o nome enviado.2. O identificador deve ser gerado automaticamente.3. A API deve retornar HTTP 201 com os dados do autor criado.	Disponibiliza POST /api/authors. O autor é um registro de identificação da autoria, sem credenciais de acesso.
US10 Consultar autor e seus tutoriais	Como responsável pelo catálogo, quero consultar um autor e seus tutoriais, para identificar os materiais atribuídos a ele.	1. A consulta deve localizar o autor pelo identificador.2. Quando encontrado, deve retornar HTTP 200 com seus dados e os tutoriais associados.3. Quando não encontrado, deve retornar HTTP 404.	Disponibiliza GET /api/authors/{id} e permite consultar a relação entre um autor e seus tutoriais.
US11 Definir o autor de um tutorial	Como responsável pelo catálogo, quero associar ou substituir o autor de um tutorial, para manter correta a atribuição de autoria.	1. O tutorial e o autor devem existir.2. A associação deve manter, no máximo, um autor por tutorial.3. Caso exista um autor anterior, o vínculo deve ser substituído.4. Deve retornar HTTP 200 em caso de sucesso ou HTTP 404 quando um dos registros não existir.	Disponibiliza POST /api/tutorials/{tutorialId}/author/{authorId}. Um mesmo autor pode estar associado a vários tutoriais.
US12 Cadastrar curso	Como responsável pelo catálogo, quero cadastrar um curso, para utilizá-lo como	1. O cadastro deve armazenar o nome enviado.2. O identificador deve ser gerado automaticamente.3.	Disponibiliza POST /api/courses. A vinculação dos tutoriais ocorre em uma operação separada.

	agrupamento de tutoriais relacionados.	A API deve retornar HTTP 201 com os dados do curso criado.	
US13 Consultar curso	Como responsável pelo catálogo, quero consultar um curso pelo identificador, para verificar seus dados cadastrais.	1. A consulta deve localizar o curso pelo identificador.2. Quando encontrado, deve retornar HTTP 200 com identificador e nome.3. Quando não encontrado, deve retornar HTTP 404.	Disponibiliza GET /api/courses/{id}. A resposta atual não inclui a coleção de tutoriais do curso.
US14 Vincular tutorial a curso	Como responsável pelo catálogo, quero associar um tutorial a diferentes cursos, para reaproveitar o material em vários agrupamentos.	1. O tutorial e o curso devem existir.2. A associação deve preservar os vínculos do tutorial com outros cursos.3. Em caso de sucesso, deve retornar HTTP 200 com o tutorial atualizado e seus cursos.4. Se um dos registros não existir, deve retornar HTTP 404.	Disponibiliza POST /api/tutorials/{tutorialId}/courses/{courseId} e utiliza o relacionamento de muitos para muitos entre tutoriais e cursos.
US15 Remover tutorial de um curso	Como responsável pelo catálogo, quero remover o vínculo entre um tutorial e um curso, para reorganizar os conteúdos mantendo seus cadastros.	1. O tutorial, o curso e a associação devem existir.2. A operação deve remover somente o vínculo, preservando os registros.3. Em caso de sucesso, deve retornar HTTP 204 sem corpo.4. Se um dos registros ou o vínculo não existir, deve retornar HTTP 404.	Disponibiliza DELETE /api/tutorials/{tutorialId}/courses/{courseId} para alterar a composição dos cursos.

US16 Registrar detalhes do tutorial	Como responsável pelo catálogo, quero registrar conteúdo e tempo estimado para um tutorial, para complementar suas informações.	1. O tutorial informado deve existir.2. A operação deve armazenar conteúdo e tempo estimado em minutos, vinculando os detalhes ao tutorial.3. Em caso de sucesso, deve retornar HTTP 201 com os detalhes cadastrados.4. Se o tutorial não existir, deve retornar HTTP 404.	Disponibiliza POST /api/tutorials/{tutorialId}/detail. O modelo estabelece uma relação de um para um entre tutorial e registro de detalhes.
US17 Consultar detalhes do tutorial	Como responsável pelo catálogo, quero consultar o conteúdo e o tempo estimado de um tutorial, para compreender o material e o tempo previsto para sua realização.	1. A consulta deve procurar os detalhes pelo identificador do tutorial.2. Quando encontrados, deve retornar HTTP 200 com conteúdo e tempo estimado.3. Quando não houver detalhes para o identificador informado, deve retornar HTTP 404.	Disponibiliza GET /api/tutorials/{tutorialId}/detail para consultar as informações complementares do material.